环境变量的来源、原理与应用

2016-10-02 2626 编译 链接 装载 库 环境变量 0

简介

本文结合操作系统的原理，以程序装载、运行的角度来介绍环境变量的原理，主要考虑环境变量在操作系统中的位置，环境变量如何加载，环境变量为什么会变化，应当如何应用环境变量等问题，以及一些需要注意的地方。

开发环境

本文中的介绍主要基于Ubuntu 16.04.1；在涉及到需要调试的C语言代码时，为方便起见使用macOS Sierra。

引言

对于不知道环境变量作用原理的人来说，环境变量似乎是一个很模糊的存在，直观上就像是我在一个指定的文件中添加我需要的环境变量（比如给 `PATH` 加个路径，创建一个如 `AA=bb` 的环境变量），然后就可以直接使用这个环境变量了，就像一个全局变量一样。一个最典型的例子就是bash，如果我们希望执行某个程序时不用输入完整路径，通常就会在 `~/.bashrc` 中的 `PATH` 中加上相应的路径，这样以后就只用输入程序名了。

对bash一个直观的理解就是，当输入一个命令时，bash会在环境变量 `PATH` 列出的所有路径中寻找此命令，找到后执行此命令。这样的理解不能算正确但也不能算错误，深入点考虑，从操作系统角度来说，bash就是一个进程，当bash进程启动后，就再也不能向操作系统“要”任何数据了（事实上对于操作系统而言也没有“要”这个概念），但如果你在bash中打印此时的所有环境变量，你会发现环境变量的数目远比 `~/.bashrc` 等中手动添加的环境变量要多，那多余的时从哪来的呢？

进程的环境变量

每一个进程都有自己的环境变量。更详细地说，进程在创建时，环境变量就会压入栈中。拿一个简单的C语言程序作为例子。

打印环境变量

C代码：

```c
#include <stdio.h>

extern char **environ;

int main(int argc, const char *argv[]) {
  int i = 0;
  while (environ[i]) {
    printf("%s\n", environ[i]);
    i++;
  }
}
```

运行输出：

```
XDG_SESSION_ID=15
TERM=xterm-256color
SHELL=/bin/bash
SSH_CLIENT=192.168.1.2 57959 22
SSH_TTY=/dev/pts/0
USER=ubuntu
MAIL=/var/mail/ubuntu
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
PWD=/home/ubuntu
LANG=en_US.UTF-8
SHLVL=1
HOME=/home/ubuntu
LOGNAME=ubuntu
SSH_CONNECTION=192.168.1.2 57959 192.168.1.11 22
LC_CTYPE=zh_CN.UTF-8
LESSOPEN=| /usr/bin/lesspipe %s
XDG_RUNTIME_DIR=/run/user/1000
LESSCLOSE=/usr/bin/lesspipe %s %s
_=./out
```

打印出来的这些就是这个小程序运行时进程中的环境变量。 `**environ` 就是C语言运行时环境提供的对进程环境变量的访问指针，指向一个个环境变量字符串。

环境变量在进程哪个地方

`**environ` 似乎不太有说服力，下面来仔细看看环境变量在哪。

```c
#include <stdio.h>

extern char **environ;

int main(int argc, const char *argv[]) {
  printf("environment variables:\n");
  int i = 0;
  while (environ[i]) {
    printf("%p\t%s\n", environ[i], environ[i]);
    i++;
  }

  printf("argv:\n");
  for (int i = 0; i < argc; i++) {
    printf("%p\t%s\n", argv[i], argv[i]);
  }
}
```

编译为 `out`，执行

```
./out first second third
```

运行输出：

```
environment variables:
0x7e8f8881    XDG_SESSION_ID=15
...
0x7e8f8e94    PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
0x7e8f8efc    PWD=/home/ubuntu
0x7e8f8f0d    LANG=en_US.UTF-8
...
0x7e8f8fee    _=./out
argv:
0x7e8f8868    ./out
0x7e8f886e    first
0x7e8f8874    second
0x7e8f887b    thrid
```

先看熟悉的 `argv`， `argv` 是从 `bash` 传来的参数，在调用 `main()` 时作为实参压入栈中；栈向内存地址减小的方向生长，所以这些参数从右向左依次入栈。再来看看环境变量的内存地址，其值都比 `argv` 大，且按顺序依次增大；意味着环境变量在 `argv` 之前就入栈，并且在栈中环境变量和 `argv` 紧邻。

形象地来说， `main()` 执行时的栈：

...	0	env n	...	env 0	0	arg n	...	arg 0	argc	...
		*l_*	*env*	*_l*		*l_*	*argv*	*_l*		
->	->	->	->	->	stack	->	->	->	->	->

或者还可以将

```c
int main (int argc, char *argv[])
```

理解为

```c
int main (int argc, char *argv[], char *envp[])
```

即调用 `main()` 时实际还传递了环境变量数组。

环境变量怎么使用

在C中通过 `getenv()` 就可以获得指定环境变量的值，非常方便。

```c
#include <stdio.h>
#include <stdlib.h>

int main(int argc, const char *argv[]) {
  char *home = getenv("HOME");

  printf("Your home directory is %s.\n", home);

  return 0;
}
```

运行输出：

```
Your home directory is /home/ubuntu.
```

小结

每个进程都拥有自己的环境变量，且不同进程间环境变量不受干扰（修改一个进程的环境变量不会影响到其他进程，即使是同一程序）。由于 `main()` 的执行伴随进程始终，因此环境变量在程序执行过程中不会出栈，即一直存在。甚至，由于环境变量所占栈空间大小不能更改，因此程序运行过程中并不能“实际意义上”添加环境变量。

环境变量从哪来

继承

进程的环境变量继承自其父进程。父进程在创建子进程时，可以修改子进程的环境变量（修改、添加或删除），但一旦子进程创建完毕，子进程和父进程的环境变量便不再有任何联系。

用简单的C语言例子来说明。

例一

父进程：

```c
#include <stdio.h>
#include <unistd.h>

extern char **environ;

void show_env() {
  printf("environment variables:\n");
  int i = 0;
  while (environ[i]) {
    printf("%p\t%s\n", environ[i], environ[i]);
    i++;
  }
}

int main(int argc, const char *argv[]) {
  printf("parent process:\n");
  show_env();
  if (fork() == 0) {
    execl("./child", "child", NULL);
  }

  return 0;
}
```

子进程：

```c
#include <stdio.h>

extern char **environ;

void show_env() {
  printf("environment variables:\n");
  int i = 0;
  while (environ[i]) {
    printf("%p\t%s\n", environ[i], environ[i]);
    i++;
  }
}

int main(int argc, const char *argv[]) {
  printf("child process\n");
  show_env();
}
```

父进程首先打印出自己的环境变量，然后 `fork()` 创建一个克隆的新进程，再通过 `exec()` 执行新的程序child。

运行输出：

```
parent process:
environment variables:
0x7ed19881    XDG_SESSION_ID=32
```

```
0x7ed19893    TERM=xterm-256color
0x7ed198a7    SHELL=/bin/bash
0x7ed198b7    SSH_CLIENT=192.168.1.2 50653 22
0x7ed198d7    SSH_TTY=/dev/pts/0
0x7ed198ea    USER=ubuntu
0x7ed19e7e    MAIL=/var/mail/ubuntu
0x7ed19e94    PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
...

child process
environment variables:
0x7eec887f    XDG_SESSION_ID=32
0x7eec8891    TERM=xterm-256color
0x7eec88a5    SHELL=/bin/bash
0x7eec88b5    SSH_CLIENT=192.168.1.2 50653 22
0x7eec88d5    SSH_TTY=/dev/pts/0
0x7eec88e8    USER=ubuntu
0x7eec8e7c    MAIL=/var/mail/ubuntu
0x7eec8e92    PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
...
```

可以看到子进程和父进程的环境变量完全相同，子进程继承了父进程的环境变量。

例二

修改父进程，在 `fork()` 出新的进程后，修改环境变量 `PATH` ，增加一个新的环境变量 `PENGUIN=BEAR` 。

```c
#include <stdio.h>
#include <unistd.h>

extern char **environ;

void show_env() {
  printf("environment variables:\n");
  int i = 0;
  while (environ[i]) {
    printf("%p\t%s\n", environ[i], environ[i]);
    i++;
  }
}

int main(int argc, const char *argv[]) {
  printf("parent process:\n");
  show_env();
  if (fork() == 0) {
    setenv("PATH", "wrong", 1);
    putenv("PENGUIN=BEAR");
    execl("./child", "child", NULL);
  }

  return 0;
}
```

运行输出：

```
parent process:
environment variables:
0x7e81d881    XDG_SESSION_ID=32
0x7e81d893    TERM=xterm-256color
0x7e81d8a7    SHELL=/bin/bash
0x7e81d8b7    SSH_CLIENT=192.168.1.2 50653 22
0x7e81d8d7    SSH_TTY=/dev/pts/0
0x7e81d8ea    USER=ubuntu
0x7e81de7e    MAIL=/var/mail/ubuntu
0x7e81de94    PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
...

child process
0x7e80d8cf    XDG_SESSION_ID=32
0x7e80d8e1    TERM=xterm-256color
0x7e80d8f5    SHELL=/bin/bash
0x7e80d905    SSH_CLIENT=192.168.1.2 50653 22
0x7e80d925    SSH_TTY=/dev/pts/0
0x7e80d938    USER=ubuntu
0x7e80decc    MAIL=/var/mail/ubuntu
0x7e80dee2    PATH=wrong

```

```
...
0x7e80dfe7    PENGUIN=BEAR
```

可以看到子进程的环境变量 `PATH` 被修改为了 `wrong` ，并且多了一个环境变量 `PENGUIN=BEAR` ，这些环境变量都一起最先压入栈中。即父进程在创建子进程时，可以修改子进程的环境变量，子进程创建完毕调用 `main()` 时修改后的环境变量会首先压入栈中。

从整体看

既然子进程的环境变量是继承自父进程的，那么父进程的环境变量又是从哪来的呢？

答案很简单：从父进程的父进程。在操作系统中，除特殊的第一个进程外，每一个进程都是由另一个进程创造，操作系统中的进程有确定的父子关系（像树的结构一样），不存在凭空出现的进程。所以操作系统中的每一个进程的环境变量的来源也很容易解释，环境变量通过继承和修改一步一步由父进程传递给子进程。

在Linux中，进程继承得到的环境变量保存在 `/proc/<pid>/environ` 中（不包括进程运行中修改的环境变量），借着这个我们来简单分析下上述例子的环境变量继承关系。

首先再把父进程的环境变量列举一下：

```
XDG_SESSION_ID=15
TERM=xterm-256color
SHELL=/bin/bash
SSH_CLIENT=192.168.1.2 57959 22
SSH_TTY=/dev/pts/0
USER=ubuntu
MAIL=/var/mail/ubuntu
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games:/snap/bin
PWD=/home/ubuntu
LANG=en_US.UTF-8
SHLVL=1
HOME=/home/ubuntu
LOGNAME=ubuntu
SSH_CONNECTION=192.168.1.2 57959 192.168.1.11 22
LC_CTYPE=zh_CN.UTF-8
LESSOPEN=| /usr/bin/lesspipe %s
XDG_RUNTIME_DIR=/run/user/1000
LESSCLOSE=/usr/bin/lesspipe %s %s
_=./out
```

我们知道这个进程是通过bash创建得到的（我们使用的交互命令行就是bash，在bash中输入命令运行程序，当然是从bash创建的了），通过 `ps -A` 找到bash所在的进程号，看看bash的环境变量：

```
LANG=en_US.UTF-8
LC_CTYPE=zh_CN.UTF-8
USER=ubuntu
LOGNAME=ubuntu
HOME=/home/ubuntu
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/games
MAIL=/var/mail/ubuntu
SHELL=/bin/bash
SSH_CLIENT=192.168.1.2 50653 22
SSH_CONNECTION=192.168.1.2 50653 192.168.1.11 22
SSH_TTY=/dev/pts/0
TERM=xterm-256color
XDG_SESSION_ID=32
XDG_RUNTIME_DIR=/run/user/1000
```

可以看到bash在创建我们的程序的进程时，又向环境变量中添加了新的环境变量。

因为我们是通过ssh连接Ubuntu再使用bash运行的程序，所以这个bash又是由ssh创建的（ssh的bash和/bin/bash也是有区别的），再来看看sshd的环境变量：

```
LANG=en_US.UTF-8
PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin
NOTIFY_SOCKET=/run/systemd/notify
SSHD_OPTS=
```

可以看到sshd进程本身在创建时继承得到的环境变量很少，但当bash创建时从sshd继承到的环境变量又很多，也就是说sshd本身在运行时添加了一些环境变量，如关键的 `USER` ， `HOME` 等环境变量就是由sshd添加，同时也对 `PATH` 进行了修改。

去看OpenSSH的源码也能佐证这一观点，设置环境变量集中在 `session.c` 中：

```c
...
if ((cp = getenv("AUTHSTATE")) != NULL)
  child_set_env(&env, &envsize, "AUTHSTATE", cp);
read_environment_file(&env, &envsize, "/etc/environment");
...
child_set_env(&env, &envsize, "USER", pw->pw_name);
```

```c
child_set_env(&env, &envsize, "LOGNAME", pw->pw_name);
#ifdef _AIX
child_set_env(&env, &envsize, "LOGIN", pw->pw_name);
#endif
child_set_env(&env, &envsize, "HOME", pw->pw_dir);
#ifdef HAVE_LOGIN_CAP
    if (setusercontext(lc, pw, pw->pw_uid, LOGIN_SETPATH) < 0)
        child_set_env(&env, &envsize, "PATH", _PATH_STDPATH);
    else
        child_set_env(&env, &envsize, "PATH", getenv("PATH"));
#else /* HAVE_LOGIN_CAP */
...
child_set_env(&env, &envsize, "MAIL", buf);

/* Normal systems set SHELL by default. */
child_set_env(&env, &envsize, "SHELL", shell);

if (getenv("TZ"))
  child_set_env(&env, &envsize, "TZ", getenv("TZ"));
...
```

如bash中的诸多环境变量就是通过这些代码设置的。

sshd作为一个service应该是由systemd创建的（从Ubuntu 15.04开始引入了systemd），当然从systemd到sshd也可能还有其他一些机制参与，但这里为了简单起见直接分析systemd了。systemd就是那个很特殊的第一个进程，其进程ID号为1，看看systemd的环境变量：

```
HOME=/
init=/sbin/init
recovery=
TERM=linux
drop_caps=
PATH=/sbin:/usr/sbin:/bin:/usr/bin
PWD=/
rootmnt=/root
```

可以看到sshd从systemd继承得到的环境变量中多了 `LANG` ， `PATH` 也进行了修改。我们主要关注 `LANG` ，实际上systemd对环境变量作的最主要的修改就是确定 `LANG` ，看看systemd的源码 `src/core/locale-setup.c` ：

```c
r = parse_env_file("/etc/default/locale", NEWLINE,
     "LANG",              &variables[VARIABLE_LANG],
     "LANGUAGE",          &variables[VARIABLE_LANGUAGE],
     "LC_CTYPE",          &variables[VARIABLE_LC_CTYPE],
     "LC_NUMERIC",        &variables[VARIABLE_LC_NUMERIC],
     "LC_TIME",           &variables[VARIABLE_LC_TIME],
     "LC_COLLATE",        &variables[VARIABLE_LC_COLLATE],
     "LC_MONETARY",       &variables[VARIABLE_LC_MONETARY],
     "LC_MESSAGES",       &variables[VARIABLE_LC_MESSAGES],
     "LC_PAPER",          &variables[VARIABLE_LC_PAPER],
     "LC_NAME",           &variables[VARIABLE_LC_NAME],
     "LC_ADDRESS",        &variables[VARIABLE_LC_ADDRESS],
     "LC_TELEPHONE",      &variables[VARIABLE_LC_TELEPHONE],
     "LC_MEASUREMENT",    &variables[VARIABLE_LC_MEASUREMENT],
     "LC_IDENTIFICATION", &variables[VARIABLE_LC_IDENTIFICATION],
     NULL);
```

systemd会读取 `/etc/default/locale` 来设置语言相关的环境变量，这也就是 `LANG=en_US.UTF-8` 的由来了。

仔细梳理一下会发现我们的程序用到的环境变量就是在这些父进程中一步步继承得到，脉络很清晰。

注意：可能你会问**“systemd**是第一个进程，但**systemd**也继承了环境变量，那这个环境变量是从哪来的呢？”我找到好多资料但都没有非常明确的解释，这里姑且理解为系统启动时由启动引导程序生成的。还望不吝赐教！

小结

通过上述例子或许还能解开一个疑惑：Linux中众多的设置环境变量的文件（如/etc/environment，~/.bashrc），我到底该修改哪个呢？

可以发现，这些设置环境变量的文件实际是不同的进程在创建时固定读取的，如systemd会读取 `/etc/default/locale` ，sshd会读取 `/etc/environment` ，bash会读取 `~/.bashrc` ；而进程间的父子关系又决定了这些环境变量的加载时机和作用范围，如systemd作为所有进程的父进程，修改 `/etc/default/locale` 会作用到所有进程中，bash仅为当前用户提供交互，所以修改 `~/.bashrc` 只会对当前的bash有效。当将一个个配置文件对应到相应的进程，理顺父子关系后，环境变量的配置问题便很容易解决了。

总结

整个写下来似乎有些繁杂，但我觉得理解环境变量最重要的两点就是环境变量以进程为单位，子进程继承父进程的环境变量。并且对于操作系统来说，环境变量并没有特殊的地位，只是一个执行程序时默认传输的参数而已。环境变量不会无缘无故地产生，也不会无缘无故地消失，一切变化都是在代码的控制下完成。

参考

- 俞甲子, 潘爱民. 程序员的自我修养: 链接, 装载与库[M]. 电子工业出版社, 2009.
- The GNU C Library: Program Arguments
- Linux: where are environment variables stored? - Stack Overflow
- exec
- Why does an SSH remote command get fewer environment variables then when run manually? - Stack Overflow
- openssh-portable/session.c at master · openssh/openssh-portable · GitHub
- systemd/locale-setup.c at master · systemd/systemd · GitHub
- EnvironmentVariables - Community Help Wiki
- SystemdForUpstartUsers - Ubuntu Wiki
- 走进Linux之systemd启动过程-技术 学习|Linux.中国-开源社区